DEPARTMENT OF MATHEMATICS & COMPUTER SCIENCE

Faculty of Science and Technology · National University of Lesotho

CS3520

Computer Organisation and Architecture I

Week 8 — Pipelining I: The Pipelined Datapath, Hazards & Forwarding

Mr K. O. Setetemela

Course Coordinator · Department of Mathematics & Computer Science

Academic Year 2026/2027 · Semester A

Recovering the Wasted Time

Weeks 6–7: our single-cycle machine works, and has CPI = 1 — but its clock is hostage to the slowest instruction, and most hardware sits idle most of the time. Pipelining puts that idle hardware to work.

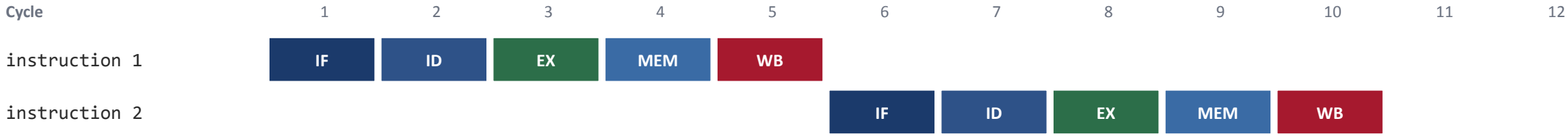
1	**The Idea of Pipelining**	Overlap, not acceleration
2	**The Five RISC-V Stages**	IF, ID, EX, MEM, WB
3	**Pipeline Speedup**	What it gains — and what it does not
4	**The Pipelined Datapath**	Pipeline registers between every stage
5	**Hazards**	Three ways the ideal breaks down
6	**Forwarding**	Using a result before it is written
7	**The Load-Use Case**	When forwarding is not enough

THE CORE IDEA

Overlap, Not Acceleration

Nothing in a pipeline runs faster. More things run at once — and that is an entirely different kind of win.

WITHOUT PIPELINING — one instruction finishes before the next begins



PIPELINED — a new instruction starts every cycle



Each individual instruction still takes five cycles — latency is unchanged. What improves is throughput: how many finish per unit time. Hold that distinction; it is the most misunderstood point in the topic.

THE RISC-V PIPELINE

Five Stages, One Step Each

IF

Instruction Fetch

Read the instruction at the PC; compute PC + 4

ID

Instruction Decode

Decode the instruction; read the register operands

EX

Execute

ALU operation, address calculation, or comparison

MEM

Memory Access

Read or write data memory — loads and stores only

WB

Write Back

Write the result into the destination register

Why five, and why these five? Because the RISC-V instruction set was designed to permit exactly this split: fixed 32-bit instructions make IF uniform, regular formats put the register fields in fixed places so ID needs no arithmetic, and load–store addressing means an address can be computed in EX and used in MEM. A less regular ISA would not divide so cleanly.

WORKED EXAMPLE

How Much Does Pipelining Buy?

Suppose the five stages take 200, 100, 200, 200 and 100 ps. Compare a single-cycle machine with a pipelined one.

Single-cycle  **800 ps** *the sum of all five stages*

Pipelined  **200 ps** *the slowest single stage*

```
Speedup = 800 / 200 = 4x

# but the pipeline has FIVE stages,
# so why is the speedup only 4x?
```

Because the stages are unbalanced. The clock is set by the slowest stage, so the 100 ps stages waste 100 ps each, every cycle. Perfect speedup needs perfectly equal stages — which real designs never quite achieve.

Note also that this ignores the time to fill and drain the pipeline. Over millions of instructions that cost vanishes — over five, it dominates.

THE HARDWARE

The Pipelined Datapath

Same functional units as Week 6 — but with registers between every pair of stages, holding everything the next stage will need.



pipeline registers

What must a pipeline register hold?

Everything a later stage still needs — and that includes things the current stage has finished with. The destination register number, for instance, is read in ID but not used until WB, four stages later. It must therefore be carried the whole way. Control signals travel with it: each stage's signals are generated once, in ID, then handed down the pipeline alongside the data they govern.

WHEN THE IDEAL BREAKS

Three Kinds of Hazard

A hazard is any situation that prevents the next instruction from starting in the next cycle.



Structural

Two instructions need the same hardware in the same cycle.

Solved by design: separate instruction and data memories.



Data

An instruction needs a result that an earlier instruction has not yet written.

Solved by forwarding — and sometimes a stall.



Control

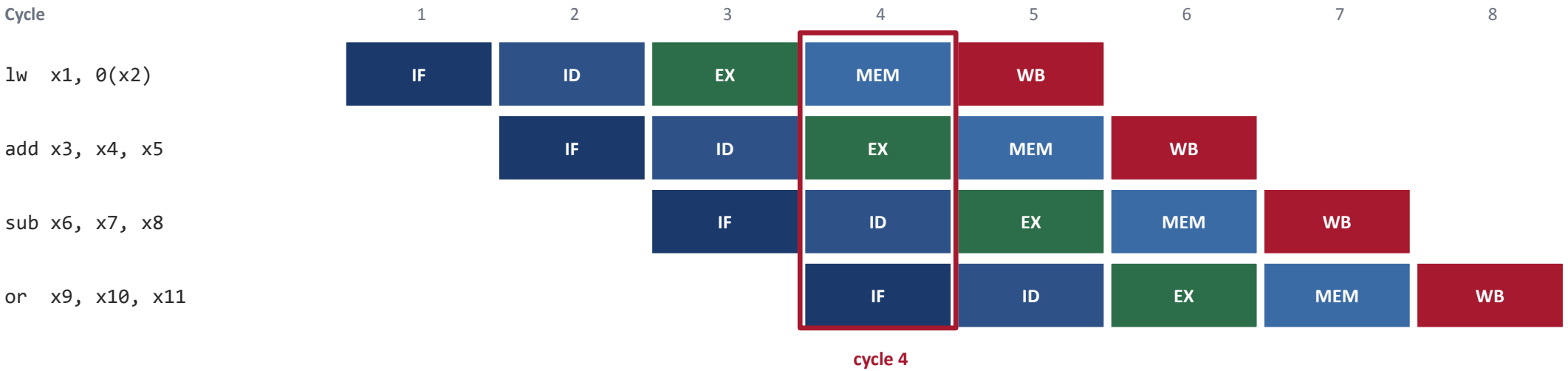
The next instruction to fetch depends on a branch not yet resolved.

Solved by prediction — which is Week 9.

HAZARD 1

Structural Hazards

Imagine a machine with one memory serving both instructions and data. Watch what happens in cycle 4.



In cycle 4, lw is in MEM reading data, while or is in IF reading an instruction.

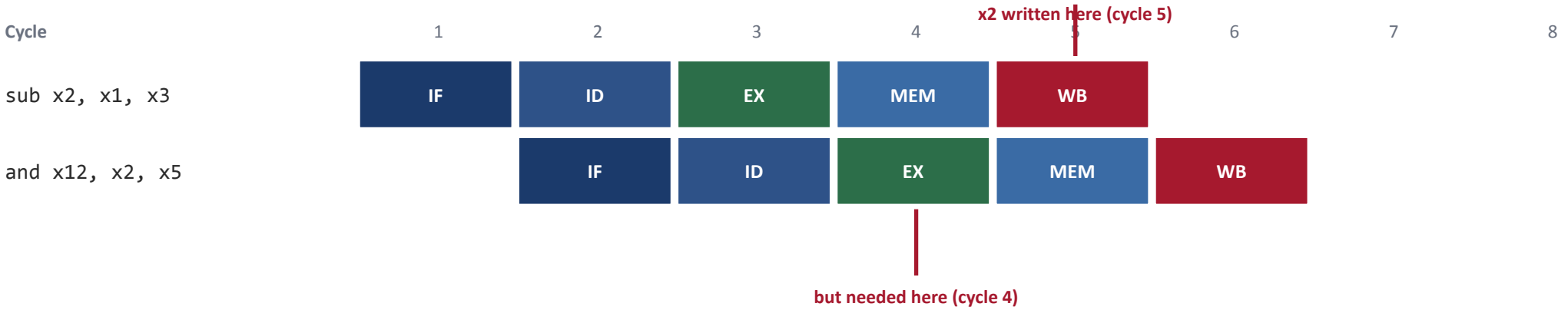
One memory cannot serve both. The pipeline would have to stall — inserting a bubble — every time a load or store appeared, which in real code is roughly one instruction in three. That is why pipelined machines have separate instruction and data memories (in practice, separate caches). This hazard is designed away rather than handled at run time.

HAZARD 2

Data Hazards

```
sub x2, x1, x3
and x12, x2, x5
```

and needs x2 — but sub has not written it yet.

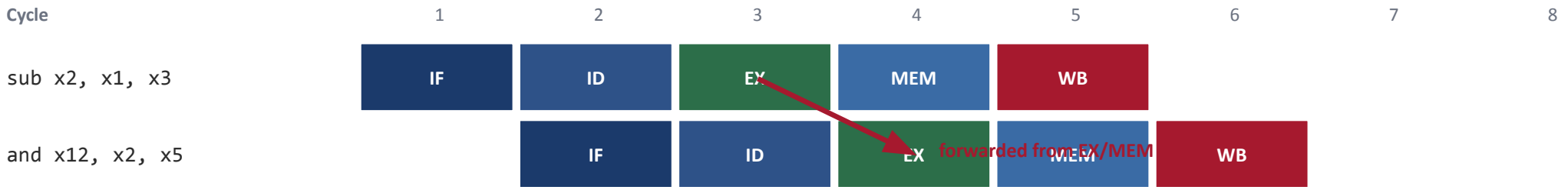


The value is needed one cycle before it is written. But look again at the first diagram: the ALU produced the answer at the end of cycle 3. It exists — it is simply sitting in a pipeline register instead of the register file. If the hardware could reach in and take it, no one would have to wait.

THE SOLUTION

Forwarding (Bypassing)

Take the result from wherever it already exists, rather than waiting for it to reach the register file.



DETECTING IT — THE FORWARDING UNIT

```
EX hazard   if (EX/MEM.RegWrite and EX/MEM.rd != x0
                and EX/MEM.rd == ID/EX.rs1)   ForwardA = 10

MEM hazard  if (MEM/WB.RegWrite and MEM/WB.rd != x0
                and MEM/WB.rd == ID/EX.rs1)   ForwardB = 01
```

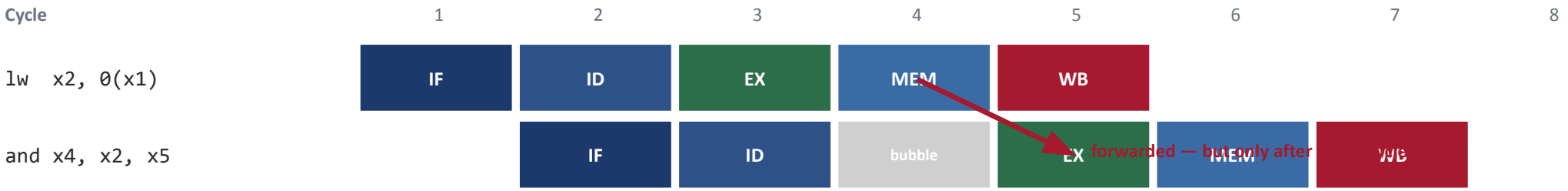
Three conditions, every time: the earlier instruction must actually write a register, that register must not be x0, and it must be the one we need. Miss any one and the processor forwards a value it should not.

WHEN FORWARDING IS NOT ENOUGH

The Load-Use Hazard

```
lw  x2, 0(x1)
and x4, x2, x5
```

A load produces its value in MEM — one stage later than the ALU does.



Why a stall is unavoidable

You cannot forward backwards in time. The data arrives at the end of cycle 4; the instruction that wants it enters EX in cycle 4. The hardware detects this in ID and freezes the pipeline for one cycle.

What a compiler can do

Reorder the code. If an independent instruction can be moved into the gap, the bubble disappears and the work gets done for free. This is why compiler writers must understand the pipeline — and why the same source code performs differently on different machines.

THINK ABOUT IT

Forwarding makes the pipeline correct. It does not make it simple — it adds comparators, multiplexers and wires to every stage.

Each hazard we solve costs hardware, power and design effort. At what point would you stop adding pipeline stages? And what happens to the cost of a single mistaken branch as the pipeline grows deeper?

Discuss with a neighbour for two minutes before we continue.

THIS WEEK

Lab 7 & Project Milestone 1

Visualise the pipeline

Load your Lab 2 programs into Ripes and watch all five stages in flight.

Find a real hazard

Locate a data hazard in your own code and identify the forwarding path that resolves it.

Force a stall

Write a load-use sequence, observe the bubble, then reorder the code to remove it.

PROJECT MILESTONE 1

Design review demonstrations, in your lab slot this week. Worth 3%.

- Your processor design, presented as a block diagram
- The instruction subset you have chosen to support
- Evidence that something runs — however small
- A clear statement of what remains

Every team member will be asked a question. Marks follow understanding, not slides.

CHECKPOINT · CLO3

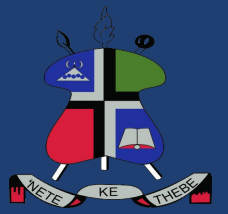
Key Takeaways

1. Explain why pipelining improves throughput but not latency, and compute the speedup for a given set of stage delays.
2. Describe what each pipeline register must carry, and why control signals travel with the data.
3. Identify structural and data hazards in a given instruction sequence.
4. State the forwarding conditions precisely, and explain why a load-use hazard still costs one cycle.

COMING UP — WEEK 9

Pipelining II — Control Hazards & the Memory Hierarchy

Branch penalties, stalls, branch prediction — then locality and caches



Reading: Patterson & Hennessy, RISC-V Edition, §4.5–§4.7